

Input Encoders for Multi-Channel Signal Transformers: An Empirical Audit of Subspace Geometry

Ossi Lehtinen*

Ocon Oy

Abstract

Transformers consuming multi-channel scalar signals must embed C simultaneous values into one d_{model} -dimensional vector per time step. We compare eight input encoders—spanning per-channel linear projections, an orthogonality regulariser, a nonlinear MLP stem, block-partitioned concatenation, channel-independent and channel-as-token architectures, and a projected positional encoding—on a synthetic benchmark where channel identity is informative and on ETTh1 as a real-data check. At convergence the standard per-channel linear projection (`nn.Linear(C,  $d_{\text{model}}$ )`) matches every alternative we test to within seed noise. A variant that projects the sinusoidal positional encoding through a learned linear layer (LINEAR-PPE) shows a small (~ 0.05 NLL) advantage at $C=4$ on five seeds—suggestive but at the edge of what our sample resolves, and with the underlying mechanism (positional-channel orthogonalisation vs. positional subspace compression) not pinned down by the present data. A gram-matrix analysis shows the task loss drives channel projections to near-orthogonality without explicit regularisation, and learned norms are systematically larger for the outcome-relevant channels. Code: <https://github.com/OssiLehtinen/FDM-encoding>.

1 Introduction

Transformers applied to multivariate numerical signals—multi-sensor telemetry, financial factor stacks, biomedical waveforms—face a design choice at the input layer: how to embed C simultaneous scalar channels into one d_{model} -dimensional vector per time step.

Prior art. The additive embedding recipe inherited from NLP (1) projects each input to a vector and sums. When the inputs are multiple channels rather than one token-plus-position pair, this comes in two forms: a *shared* scalar projection W with per-channel additive embeddings, or a *per-channel* projection W_k —the latter being mathematically a single `nn.Linear(C,  $d_{\text{model}}$ )`. Many multivariate time-series transformers use a linear input stem (2, 3), though details vary. Alternative architectures treat each channel as a separate token (iTransformer (5)), run independent backbones per channel (PatchTST (4)), or combine both (Crossformer (6)). Orthogonality penalties on weight matrices have been used for feature decorrelation in deep networks (7).

*ossi@ocon.fi

Open questions. Despite the variety of approaches, several basic questions remain unanswered in the literature: (i) Does a shared-scalar projection with a small channel embedding actually work, or does it have a hard ceiling? (ii) Is block partitioning, an orthogonality penalty, or nonlinearity needed, or does a bare linear layer suffice? (iii) Does the task loss itself produce channel separation without explicit enforcement? (iv) Does the positional encoding interact with the channel encoding, and can this interaction be improved?

Our contribution. We compare eight encoders on a controlled synthetic benchmark designed to make channel identity informative, with supporting experiments on the public ETTh1 dataset. We find that: (i) the per-channel linear projection matches every alternative at convergence; (ii) a shared-scalar baseline has a sharp, capacity-independent ceiling; (iii) the task loss drives the learned W_k to near-orthogonal subspaces and scales their norms larger for outcome-relevant channels; and (iv) projecting the sinusoidal positional encoding through a learned linear layer (LINEAR-PPE) shows a small improvement at $C=4$ that is at the edge of what five seeds can resolve, with a mechanism (orthogonalisation vs. compression of the positional subspace) we cannot pin down from the present data.

2 Method

2.1 Model architecture

All encoders share a common backbone: a small causal transformer with $d_{\text{model}}=64$, 4 attention heads, 3 layers, feed-forward width $d_{\text{ff}}=256$, dropout 0.1, pre-LayerNorm, and a linear categorical head mapping to $K=32$ logits. The *only* component that varies across experiments is the input encoder.

2.2 Encoders

We study eight encoders. In the definitions below, $v_k(t)$ is the scalar value of channel k at time t , $\mathbf{p}(t)$ is a fixed sinusoidal positional encoding (1), and all learned parameters are randomly initialised.

sum (ablation baseline). Shared scalar projection $W \in \mathbb{R}^{d_{\text{model}}}$, learned per-channel additive embedding e_k :

$$\mathbf{h}(t) = \sum_k (W v_k(t) + e_k) + \mathbf{p}(t).$$

Total encoder capacity is $\mathcal{O}(d_{\text{model}})$ regardless of C . We use this to isolate what goes wrong when the input projection shares weights across channels.

linear. Per-channel projection W_k and bias b_k , summed:

$$\mathbf{h}(t) = \sum_k (W_k v_k(t) + b_k) + \mathbf{p}(t).$$

Equivalently $\mathbf{h}(t) = W \mathbf{v}(t) + \mathbf{b} + \mathbf{p}(t)$ with $W \in \mathbb{R}^{d_{\text{model}} \times C}$: a single `nn.Linear(C,  $d_{\text{model}}$ )`.

sum-ortho. Same as LINEAR with an auxiliary loss $L_{\text{ortho}} = \lambda \sum_{i \neq j} (W_i \cdot W_j)^2 / 2$ ($\lambda=10^{-2}$). This variant isolates the regulariser’s contribution.

linear-ppe (projected positional encoding). Same channel encoding as LINEAR, but the sinusoidal position passes through a learned linear layer:

$$\mathbf{h}(t) = \sum_k (W_k v_k(t) + b_k) + W_{\text{pos}} \mathbf{p}(t) + \mathbf{b}_{\text{pos}},$$

with $W_{\text{pos}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. This lets the optimiser rotate the positional subspace to be orthogonal to the channel subspace. Adds d_{model}^2 parameters (4,096 at $d=64$).

mlp. Two-layer MLP on the channel vector: $\mathbf{h}(t) = W_2 \text{GELU}(W_1 \mathbf{v}(t) + \mathbf{b}_1) + \mathbf{b}_2 + \mathbf{p}(t)$, with $W_1 \in \mathbb{R}^{d_{\text{model}} \times C}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. Tests whether nonlinearity helps ($\sim 9\times$ more encoder parameters than LINEAR).

concat. Per-channel projection into $d_{\text{block}}=d_{\text{model}}/C$ dims, concatenated (requires $C \mid d_{\text{model}}$): $\mathbf{h}(t) = [\mathbf{e}_0(t) \parallel \dots \parallel \mathbf{e}_{C-1}(t)] + \mathbf{p}(t)$. Channel identity is an architectural invariant.

ci (channel-independent). PatchTST-spirit (4): shared-weight causal transformer per channel; final hidden states concatenated into the head. Compute: $\mathcal{O}(C B T^2)$.

cat (channel-as-token). iTransformer-spirit (5): each (t, k) pair is a token, sequence length $C \cdot T$, causal across time, bidirectional within a time step. Attention: $\mathcal{O}((CT)^2)$; skipped at $C=16$.

2.3 Training protocol

AdamW optimiser, learning rate 3×10^{-4} with cosine decay to 1% of peak, weight decay 10^{-4} , batch size 32, gradient clip 1.0. Each configuration is run over 5 random seeds. Metrics are reported at the best validation NLL achieved during training (effective early stopping). All headline experiments use 300 epochs. The gram-matrix and encoding-space analyses (Sections 3.4–3.5) were measured at a 100-epoch training point as a representative snapshot; the qualitative claims (near-orthogonality, driver/distractor norm gap) are stable across this and the converged regime.

2.4 Datasets

Synthetic. $N=512$ time series of length $T=160$ with C channels. Each channel is a sum of three sinusoids with random frequencies in $[0.005, 0.08]$ cycles/sample plus AR(1) noise, standardised. The outcome is a fixed non-linear function of the first four channels with lags 3 and 7:

$$y_t = \tanh(s_0(t-3) s_1(t)) + 0.6 \sin(1.3 s_2(t-7)) + 0.4 \mathbb{I}[s_3(t) > 0] s_0(t),$$

binned into $K=32$ quantile bins. Channels $k \geq 4$ are independent distractors. The task is designed so that an encoder that mixes channels at the input layer cannot recover the interaction structure.

ETTh1. Electricity Transformer Temperature (hourly, 7 variates). We standardise on the train split, bin the target variate OT into 32 quantile bins, and predict the next-step bin given all 7 variates over $T=160$. The model uses $d_{\text{model}}=56$ (divisible by $C=7$), 7 heads, $d_{\text{ff}}=224$; training is otherwise identical.

Table 1: Main comparison at $C=4$, $d_{\text{model}}=64$, 300 epochs, cosine LR decay, mean $\pm$ std over 5 seeds. Best val NLL (effective early stopping). Random baseline: $\text{NLL} = \ln 32 \approx 3.47$, $\text{acc}=0.031$.

Encoder	Enc. params	Val NLL $\downarrow$	Val acc $\uparrow$
SUM	320	3.252 ± 0.010	0.091 ± 0.002
CI	2,112	3.053 ± 0.008	0.139 ± 0.002
CAT	320	2.371 ± 0.068	0.210 ± 0.014
CONCAT	128	2.183 ± 0.032	0.243 ± 0.006
MLP	4,480	2.170 ± 0.032	0.244 ± 0.010
SUM-ORTHO	512	2.170 ± 0.014	0.246 ± 0.004
LINEAR	512	2.169 ± 0.014	0.248 ± 0.004
LINEAR-PPE	4,608	2.120 ± 0.033	0.258 ± 0.009

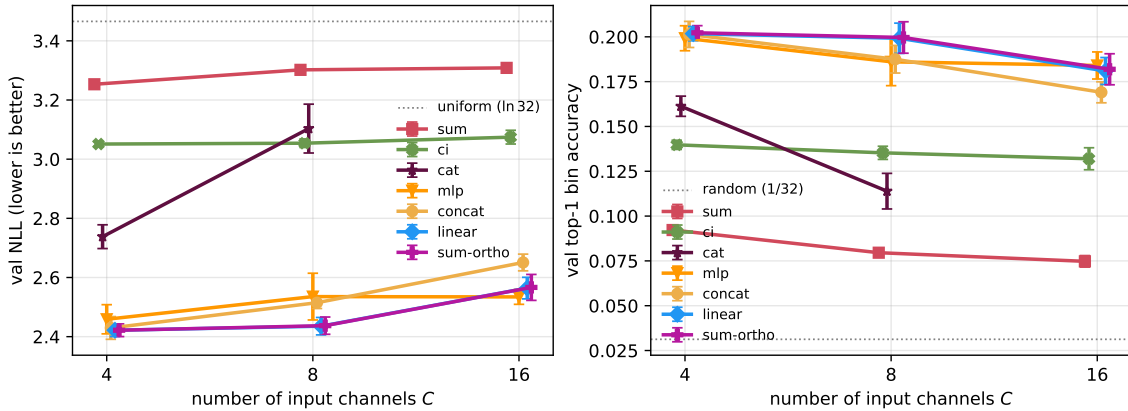


Figure 1: Scaling with input channel count C (error bars: one seed std over 5 seeds). CAT skipped at $C=16$ due to $\mathcal{O}((CT)^2)$ attention cost.

3 Results

3.1 Main comparison

Table 1 reports all eight encoders at $C=4$, $d_{\text{model}}=64$, 300 epochs with cosine LR decay (the same budget used throughout the paper). Four encoders sit within seed std of each other at the top: LINEAR, SUM-ORTHO, MLP, and CONCAT. LINEAR matches SUM-ORTHO to four decimal places—the orthogonality penalty contributes nothing at convergence. The MLP stem’s $\sim 9\times$ extra encoder parameters likewise provide no benefit. The only variant that pulls ahead is LINEAR-PPE, which projects the sinusoidal positional encoding through a learned linear layer; its ~ 0.05 NLL gap exceeds seed std but is borderline at this sample size, and we return to the question of mechanism in Section 3.8 and the Discussion. CI and CAT underperform despite nominally giving each channel more capacity. The only encoder that loses decisively is SUM, which lacks per-channel projections.

3.2 Channel-count scaling

All encoders degrade as C grows (channels $k \geq 4$ are distractors). The top tier (LINEAR, SUM-ORTHO, CONCAT) is stable across $C \in \{4, 8, 16\}$. The gap to SUM widens rather than narrows.

Table 2: Scaling with channel count. Channels 0–3 drive the outcome; additional channels are independent distractors. 100 epochs with cosine decay, mean $\pm$ std over 5 seeds. CAT is skipped at $C=16$ because its $\mathcal{O}((CT)^2)$ attention cost explodes.

C	encoder	val NLL $\downarrow$	val acc $\uparrow$
4	SUM	3.253 ± 0.007	0.092 ± 0.003
	CI	3.051 ± 0.008	0.140 ± 0.003
	CAT	2.738 ± 0.040	0.161 ± 0.006
	MLP	2.459 ± 0.049	0.199 ± 0.007
	concat	2.429 ± 0.038	0.201 ± 0.007
	linear	2.422 ± 0.021	0.202 ± 0.004
	sum-ortho	2.422 ± 0.021	0.202 ± 0.004
8	SUM	3.302 ± 0.005	0.080 ± 0.002
	CAT [†]	3.103 ± 0.083	0.114 ± 0.010
	CI	3.054 ± 0.014	0.135 ± 0.004
	MLP	2.535 ± 0.079	0.186 ± 0.013
	CONCAT	2.515 ± 0.020	0.187 ± 0.008
	linear	2.435 ± 0.029	0.199 ± 0.008
	sum-ortho	2.437 ± 0.029	0.200 ± 0.009
16	SUM	3.309 ± 0.004	0.075 ± 0.003
	CI	3.074 ± 0.023	0.132 ± 0.006
	CONCAT	2.651 ± 0.028	0.169 ± 0.006
	linear	2.564 ± 0.037	0.181 ± 0.008
	sum-ortho	2.566 ± 0.044	0.182 ± 0.009
	MLP	2.534 ± 0.025	0.184 ± 0.008

[†] CAT at $C>4$ uses the earlier causal-within-time mask; the corrected bidirectional-within-time mask was applied only to the $C=4$ rerun.

3.3 Model-width scaling

Table 3 varies d_{model} over $\{64, 128, 256\}$ at $C=4$. The SUM ceiling is sharp: $15\times$ more parameters move NLL by less than 0.02. CONCAT scales strongly to $d=128$ before mild overfitting at $d=256$.

3.4 Gram-matrix analysis: spontaneous orthogonality

LINEAR has no mechanism pushing the W_k toward orthogonality; SUM-ORTHO adds an explicit penalty. Both produce the same NLL to four decimals. We measure the cosine similarity between all pairs (W_i, W_j) with $i \neq j$ at the end of training.

Without any penalty the learned W_k are already near-orthogonal (mean off-diagonal cosine 0.054). The regulariser tightens this to 0.021 but does not improve downstream NLL. The task loss itself provides enough gradient pressure to separate the per-channel projections once they exist as free parameters.

3.5 Encoding space allocation

Beyond separating channels, does the model allocate more encoding space to channels that matter more? We measure $\|W_k\|$ norms and per-channel variance fractions $\text{Var}(W_k \cdot v_k) / \sum_j \text{Var}(W_j \cdot v_j)$

Table 3: Scaling with d_{model} at $C=4$. $d_{\text{ff}}=4d_{\text{model}}$, heads=4 (so d_{head} grows). Mean $\pm$ std over 5 seeds, 100 epochs with cosine decay. SUM shows its structural ceiling: $15\times$ more parameters barely move it. CONCAT scales strongly but begins to overfit at $d_{\text{model}}=256$. LINEAR and SUM-ORTHO were run only at $d_{\text{model}}=64$ (see Table 1) and are omitted here.

d_{model}	encoder	total params	val NLL $\downarrow$	val acc $\uparrow$
64	SUM	152,480	3.253 ± 0.007	0.092 ± 0.003
	concat	152,288	2.429 ± 0.038	0.201 ± 0.007
128	SUM	599,840	3.251 ± 0.008	0.094 ± 0.002
	concat	599,456	2.291 ± 0.009	0.223 ± 0.002
256	SUM	2,379,296	3.255 ± 0.007	0.091 ± 0.001
	concat	2,378,528	2.322 ± 0.014	0.221 ± 0.003

Table 4: Learned per-channel W_k geometry ($C=4$, mean over 5 seeds).

encoder	best val NLL	$\max_{i \neq j} \cos(W_i, W_j) $	mean $ \cos $
LINEAR	2.4215	0.152	0.054
SUM-ORTHO	2.4214	0.058	0.021

at $C \in \{4, 8, 16\}$.

Table 5: Encoding space allocation (mean over 5 seeds). Channels 0–3 drive the outcome; the rest are distractors.

	$\ W_k\ $ norm		variance fraction	
	drivers (0–3)	distractors	drivers	distractors
$C=8$	1.14	0.71	72.3%	27.7%
$C=16$	1.20	0.74	46.6%	53.4%

Driver channels receive $\sim 1.6\times$ larger seed-averaged norms than distractors at both $C=8$ and $C=16$. The aggregate variance share at $C=8$ is 72% for the four drivers; at $C=16$ the aggregate driver share is closer to half (46.6%) because there are now twelve distractors, but the per-channel variance contribution is still larger for drivers ($\sim 12\%$ each vs. $\sim 4\%$ each). We do not report per-channel error bars on the norms, and the fine-grained ordering within drivers is descriptive only; the robust claim is the driver/distractor gap, which holds at both channel counts.

3.6 Channel identifiability via linear probing

Closed-form ridge probes from the hidden state to the raw input show that all per-channel- W_k encoders achieve $R^2 \approx 1.0$ at the input layer and $R^2 \geq 0.92$ at layer 3. SUM collapses to $R^2 \approx 0.24$ at every depth.

Table 6: Linear-probe channel recovery at $C=4$, seed 0, after 100 epochs of cosine-scheduled training. Closed-form ridge probe $\mathbf{h} \mapsto \hat{x}$ from a frozen hidden state to the raw input $x \in \mathbb{R}^4$; held-out per-channel R^2 . Layer 0 is the input embedding; layer 3 is the final transformer block. All three per-channel- W_k encoders achieve near-perfect input recovery and preserve mean $R^2 \geq 0.92$ through three attention layers; SUM collapses to $R^2 \approx 0.24$ at every depth.

Encoder	Layer	R^2_{ch0}	R^2_{ch1}	R^2_{ch2}	R^2_{ch3}	mean
SUM	0 (input)	0.255	0.250	0.242	0.214	0.240
SUM	3 (final)	0.251	0.243	0.235	0.206	0.234
LINEAR	0 (input)	1.000	1.000	1.000	1.000	1.000
LINEAR	3 (final)	0.963	0.932	0.933	0.877	0.926
SUM-ORTHO	0 (input)	1.000	1.000	1.000	1.000	1.000
SUM-ORTHO	3 (final)	0.963	0.933	0.934	0.875	0.926
CONCAT	0 (input)	0.995	1.000	1.000	1.000	0.999
CONCAT	3 (final)	0.948	0.950	0.943	0.906	0.937

3.7 Channel masking

Zeroing one channel at test time (Table 7), the top-tier encoders show sharp, interpretable per-channel accuracy drops that match the outcome formula. SUM is nearly flat.

3.8 Positional projection: a small additional effect

Of the encoders compared in Table 1, only LINEAR-PPE sits outside the seed-std band of the top tier. Its ~ 0.05 NLL improvement over LINEAR is present at every checkpoint we record, not just at the end (2.225 vs. 2.271 NLL at epoch 100), so it is not a slow-burn late-training artefact. With only five seeds the gap exceeds either run’s seed std but is at the edge of what our sample resolves; we discuss possible mechanisms (positional-channel orthogonalisation vs. positional subspace compression) in the Discussion but neither is pinned down by the present data. The channel encoding itself remains a single linear layer in either case.

3.9 Convergence and wall-clock cost

A natural worry is that more involved encoders—an MLP stem, a projected positional layer, an orthogonality regulariser—might cost extra training time even when they don’t change the ceiling. From the 300-epoch traces (val NLL recorded every 20 epochs, $C=4, 5$ seeds) we extract two diagnostics: the epoch at which each run first reaches within 0.05 NLL of its own best, and the wall-clock seconds per epoch, normalised to LINEAR.

The five top-tier encoders reach within 0.05 NLL of their final value at the same trace point (~ 150 epochs) and cost essentially the same per-epoch wall time—the transformer backbone dominates, so encoder choice within this family is free. In particular, the MLP stem’s $9\times$ extra parameters do not slow convergence. The LINEAR-PPE advantage is present at every checkpoint, not just at the end (2.225 vs. 2.271 NLL at epoch 100), so it is not a slow-burn late-training artefact.

CI and CAT pay substantial wall-clock costs from their architectures, not their input encoders: $\sim 3\times$ at $C=4$ for CI (scaling with C to $\sim 14\times$ at $C=16$) and $\sim 6.5\times$ for CAT (scaling with C^2T^2 to $\sim 24\times$ at $C=8$). CAT also converges to its worse ceiling later than the top tier reaches its ceiling.

Table 7: Test-time channel ablation at $C=4$, seed 0. We zero a single channel’s input at inference; rows show the resulting val accuracy. Δ is accuracy drop from the unmasked baseline. The outcome formula uses ch0 in two interaction terms, ch1 in one, and ch2/ch3 once each with smaller marginal effect; the per-channel- W_k encoders (CONCAT, LINEAR, SUM-ORTHO) reproduce this ordering cleanly, whereas SUM, CI and CAT are much flatter.

encoder	no mask	mask ch0	mask ch1	mask ch2	mask ch3
SUM	0.093	0.078	0.087	0.090	0.093
Δ acc	—	−0.015	−0.006	−0.003	0.000
CAT	0.165	0.092	0.103	0.116	0.162
Δ acc	—	−0.073	−0.062	−0.049	−0.003
CI	0.140	0.088	0.121	0.096	0.141
Δ acc	—	−0.052	−0.019	−0.044	+0.001
CONCAT	0.200	0.091	0.103	0.119	0.188
Δ acc	—	−0.109	−0.097	−0.081	−0.012
LINEAR	0.200	0.089	0.110	0.114	0.186
Δ acc	—	−0.111	−0.090	−0.086	−0.014
SUM-ORTHO	0.201	0.089	0.109	0.115	0.187
Δ acc	—	−0.112	−0.092	−0.086	−0.014

3.10 Real-data validation: ETTh1

Table 9 confirms the synthetic ordering on ETTh1. The three per-channel- W_k encoders (LINEAR, SUM-ORTHO, CONCAT) tie at the top; MLP joins within noise. SUM fails catastrophically (NLL near $\ln 32 \approx 3.47$).

4 Discussion

linear is not new. Stacking per-channel projections W_k and summing is `nn.Linear(C,  $d_{\text{model}}$ )`—the most obvious default. Our contribution is not proposing it but auditing it: at matched parameter budget, it matches every alternative (block-partitioned, orthogonality-regularised, nonlinear, channel-independent, channel-as-token). Only the shared-projection SUM loses.

What fails about the shared-scalar baseline. SUM combines a shared W with a small additive e_k . Under $15\times$ parameter scaling its NLL moves by < 0.02 . The bottleneck is not summation—LINEAR also sums—but the shared projection, which cannot represent distinct channel directions.

Orthogonality is spontaneous. LINEAR’s W_k reach mean off-diagonal cosine ≈ 0.05 with no penalty; the regulariser tightens to ≈ 0.02 without changing NLL. The task loss provides the gradient pressure.

Nonlinearity is redundant. MLP ($9\times$ more encoder params) converges more slowly to the same NLL. A linear projection of the channel vector is sufficient.

Importance weighting is automatic. At $C=8$, driver channels get $\sim 1.6\times$ larger seed-averaged $\|W_k\|$ norms and a clear majority of the embedding variance; the same ratio holds at $C=16$. We do

Table 8: Convergence speed and wall-clock cost ($C=4$, 300 epochs, 5 seeds, single GPU). Epochs-to-target is rounded to the trace resolution of 20 epochs. Cost is per-epoch wall time normalised to LINEAR.

encoder	best NLL	epoch to NLL +0.05	relative cost
LINEAR	2.169	~ 150	$1.00\times$
SUM-ORTHO	2.170	~ 150	$1.06\times$
MLP	2.170	~ 150	$1.00\times$
CONCAT	2.183	~ 150	$0.88\times$
LINEAR-PPE	2.120	~ 150	$0.94\times$
SUM	3.252	~ 20 (plateau)	$1.00\times$
CI	3.053	~ 20 (plateau)	$3.0\times$
CAT	2.371	~ 184	$6.5\times$

Table 9: ETTh1 next-step bin prediction. 7 variates, $T=160$, $K=32$ quantile bins on the target OT (oil temperature) computed from the train split. Target is the next-step bin of OT; all 7 variates are inputs. $d_{\text{model}}=56$, 7 heads, 3 layers, $d_{\text{ff}}=224$; 100 epochs, cosine decay, 5 seeds, mean $\pm$ std of best val NLL.

encoder	val NLL $\downarrow$	val acc $\uparrow$
SUM	3.636 ± 0.060	0.008 ± 0.008
CI	0.825 ± 0.046	0.678 ± 0.023
CAT [†]	0.581	0.780
concat	0.582 ± 0.015	0.787 ± 0.008
linear	0.587 ± 0.014	0.793 ± 0.008
sum-ortho	0.588 ± 0.014	0.794 ± 0.009
mlp	0.597 ± 0.025	0.786 ± 0.005

[†] CAT single-seed result only (attention cost $\mathcal{O}((CT)^2)$; each seed takes ~ 25 min at $C=7$).

not have per-channel error bars, so the fine-grained ordering within drivers is descriptive only, but the driver/distractor gap is robust across two channel counts.

Projected positional encoding: small effect, unsettled mechanism. At $C=4$ on five seeds, LINEAR-PPE sits ~ 0.05 NLL below LINEAR—a gap that exceeds either run’s seed std but is borderline at this sample size, and we have not measured it at other channel counts in the converged regime. The mechanism is also not pinned down. Two readings are consistent with the data: (i) projecting position into a learned subspace orients it away from the channel directions; or (ii) it compresses position into fewer effective coordinates, freeing residual-stream capacity. Since fixed sinusoidal positions at $d_{\text{model}}=64$, $T=160$ already fill most directions, free W_k columns can in principle place themselves in the lowest-energy positional directions, so (ii) is plausible. A direct test—principal angles between $\text{span}(W_k)$ and the effective positional basis, plus the effective rank of that basis under LINEAR vs. LINEAR-PPE—we have not run.

Architectural baselines. CI and CAT give each channel nominally more capacity but underperform at matched parameters. CI’s bottleneck is the head combining C streams; CAT pays $\mathcal{O}((CT)^2)$

attention for no gain on this task. The wall-clock cost is substantial— $\sim 14\times$ LINEAR at $C=16$ for CI, $\sim 24\times$ at $C=8$ for CAT—and buys a worse ceiling.

No convergence-time penalty for involved encoders. MLP, SUM-ORTHO, and LINEAR-PPE reach within 0.05 NLL of their final value at the same trace point as LINEAR (~ 150 epochs) and cost the same per-epoch wall time. Within the family that uses per-channel W_k on a shared backbone, encoder choice does not change either the convergence schedule or compute budget.

LayerNorm preserves channel separation. Linear probes at layer 3 recover each channel to $R^2 \geq 0.92$; LayerNorm rescales but does not linearly mix coordinates.

Reading for non-numerical inputs. The mechanism we identify is geometric—distinct input streams need approximately orthogonal subspaces in d_{model} so the model can recover stream-of-origin—and is encoding-agnostic. For multi-field categorical inputs, bag-of-embeddings pooling is the discrete analog of SUM: field identity is irrecoverable unless the per-field embedding tables happen to land in disjoint subspaces, and an off-diagonal Gram penalty across embedding matrices would transfer directly. For multimodal tokens, per-modality projection heads (already standard) are the fix; the sharper claim is that they must remain *linearly separable in the residual stream*, not merely come from different distributions. The LINEAR-PPE finding—that projecting positional signals into a learned subspace beats adding them—is the most readily transferable: it applies equally to discrete sequence models where position competes with token semantics for the same coordinates. We do not test any of this; the variance-contribution diagnostic in particular does not translate cleanly to discrete inputs and would need a probe-based analog.

5 Limitations

- The synthetic benchmark deliberately makes channel identity informative. On tasks with exchangeable channels, SUM may not be penalised.
- Main sweep at $d_{\text{model}}=64$, $C \in \{4, 8, 16\}$; ETTh1 at $d_{\text{model}}=56$, $C=7$. The d_{model} sweep covers only SUM and CONCAT. At larger scales the overfitting dynamics may change.
- CAT is skipped at $C=16$ (synthetic) and single-seed at $C=7$ (ETTh1) for compute reasons.
- LINEAR-PPE was tested at $C \in \{4, 16\}$; the 300-epoch comparison is at $C=4$ only.
- Five seeds per configuration; marginal differences in the top tier may need more seeds to resolve.

6 Related work

Channel-as-token and channel-independent TSF. iTransformer (5) treats each variate as a token; PatchTST (4) runs independent backbones per channel; Crossformer (6) does both. These preserve per-channel capacity at the cost of longer sequences or per-channel compute. Our encoders share one token per time step.

Linear input projections in TSF. Many multivariate transformers use a linear stem (2, 3). We isolate this bare projection and compare it explicitly against the shared-scalar alternative and architectural baselines.

Orthogonality regularisation. Bansal et al. (7) use soft-orthogonality penalties for feature decorrelation. We apply the same idea to per-channel input projections and find it unnecessary: the task loss produces the same geometry.

7 Conclusion

On tasks where channel identity is informative, a per-channel linear projection (`nn.Linear(C,  $d_{\text{model}}$ )`) matches every more elaborate encoder we tested. The task loss drives the channel projections to near-orthogonality, and learned norms are larger for outcome-relevant channels. A shared-scalar projection has a capacity-independent ceiling. Nonlinearity, block partitioning, orthogonality penalties, channel-independent and channel-as-token architectures—none improve on the bare linear layer within seed noise. Projecting the sinusoidal positional encoding through a learned linear layer (LINEAR-PPE) shows a small advantage at $C=4$ that is at the edge of what five seeds resolve and whose mechanism we cannot pin down here. The practical recommendation is simple: use `nn.Linear(C,  $d_{\text{model}}$ )` for the channel encoding, and spend the complexity budget elsewhere.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. Informer: beyond efficient transformer for long sequence time-series forecasting. In *AAAI*, 2021.
- [3] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long. TimesNet: temporal 2D-variation modeling for general time series analysis. In *ICLR*, 2023.
- [4] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: long-term forecasting with transformers. In *ICLR*, 2023.
- [5] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long. iTransformer: inverted transformers are effective for time series forecasting. In *ICLR*, 2024.
- [6] Y. Zhang and J. Yan. Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *ICLR*, 2023.
- [7] N. Bansal, X. Chen, and Z. Wang. Can we gain more from orthogonality regularizations in training deep networks? In *NeurIPS*, 2018.